

Hospital Management System

Full-Stack MERN Technical Documentation & Architecture Blueprint

Stack: MongoDB, Express.js, React.js, Node.js | **Author:** Lead Full-Stack Architect | **Version:** 1.0.0

1. Executive Project Overview

The **Hospital Management System (HMS)** is a comprehensive, web-based enterprise software solution designed to streamline hospital administration, medical records management, patient appointment scheduling, billing, and doctor-patient communication. Built on the **MERN (MongoDB, Express.js, React, Node.js)** stack, this system ensures high scalability, role-based access security, and real-time operational efficiency across healthcare facilities.

Key Capabilities:

- Multi-Role Access Control (Super Admin, Doctor, Patient, Receptionist/Staff).
- Automated Appointment Booking & Queue Management.
- Electronic Health Record (EHR) & Prescription Generation.
- Billing & Automated Invoice Generation with Payment Status Tracking.
- RESTful API Architecture with Secure JWT Authentication & RBAC Middleware.

2. System Architecture & Technology Stack

The system utilizes a decoupled client-server architecture. The frontend React application interacts with the Node.js/Express backend via secure RESTful APIs, with MongoDB serving as the primary document database.

Layer	Technology Used	Purpose / Key Packages
Frontend	React.js (Vite / CRA)	Single Page Application (SPA), React Router v6, Axios, Redux Toolkit / Context API, Tailwind CSS.
Backend	Node.js & Express.js	RESTful API Engine, Middleware, JWT Authentication, Controller Logic, Express-Validator.
Database	MongoDB & Mongoose	NoSQL Document Store, Schema Validation, Data Modeling, Aggregation Pipelines.
Security	JWT & Bcrypt.js	Role-Based Access Control (RBAC), Password Hashing, HTTP-only Secure Cookies / Auth Headers.

3. Key Modules & Functional Specifications

3.1 Authentication & User Management

- **User Registration & Login:** Secure authentication supporting JWT access and refresh tokens.
- **Role-Based Authorization (RBAC):** Strict permission controls restricting access to specific endpoints based on user roles:
 - **ADMIN** Full system oversight, staff onboarding, analytics, audit logs.
 - **DOCTOR** Manage schedules, view patient histories, write prescriptions, update appointment status.
 - **PATIENT** Book appointments, view medical history, download prescriptions, view invoices.
 - **RECEPTIONIST** Onboard walk-in patients, manage appointment schedules, generate bills.

3.2 Appointment & Queue Management

- Doctor slot configuration (availability, working hours, break times).
- Real-time slot checking to prevent double-booking.
- Status workflow: Pending → Confirmed → Completed / Cancelled.

3.3 Electronic Health Records (EHR) & Prescriptions

- Digital record entry for patient vitals (Blood Pressure, Heart Rate, Temperature, Weight).
- Structured prescription builder (Medication, Dosage, Frequency, Duration, Special Instructions).
- Diagnostic test reports attachment and medical history tracking.

4. Application Directory Tree Structure

The project follows a clean modular design separating concerns between frontend and backend layers.

```
hospital-management-system/ ├── backend/ | ├── config/ | | ├── db.js # MongoDB connection setup
| | └── roles.js # User roles definition | ├── controllers/ | | ├── authController.js #
Register, Login, Token Refresh | | ├── userController.js # User profile & staff management | |
└── appointmentController.js # Booking & scheduling logic | | └── prescriptionController.js#
Medical records & EHR | | └── billingController.js # Invoicing & payments | ├── middleware/ | |
└── authMiddleware.js # JWT verification | | └── roleMiddleware.js # RBAC access checks | | └──
errorMiddleware.js # Centralized error handler | ├── models/ | | ├── User.js # Unified user
schema | | ├── Appointment.js # Appointment schema | | ├── Prescription.js # Medical records
schema | | └── Invoice.js # Billing schema | ├── routes/ | | ├── authRoutes.js | | ├──
appointmentRoutes.js | | └── medicalRoutes.js | ├── server.js # Express app initialization | └──
.env # Environment variables └── frontend/ ├── src/ | ├── assets/ # Images, icons, static files
| ├── components/ # Reusable UI elements (Navbar, Sidebar, Cards) | ├── context/ # Auth & Global
state context | ├── pages/ # Admin, Doctor, Patient Dashboards | ├── services/ # Axios API calls
| ├── App.jsx # Main App routing configuration | └── main.jsx # React entry point └──
package.json
```

5. Database Schema Designs (Mongoose Models)

5.1 User Schema (`User.js`)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: {
    type: String,
    enum: ['admin', 'doctor', 'patient', 'receptionist'],
    default: 'patient'
  },
  phone: { type: String, required: true },
  specialization: { type: String }, // Specific to Doctors
  department: { type: String },    // Specific to Doctors
  availability: [{ day: String, startTime: String, endTime: String }],
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

5.2 Appointment Schema (`Appointment.js`)

```
const mongoose = require('mongoose');

const appointmentSchema = new mongoose.Schema({
  patient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  doctor: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  appointmentDate: { type: Date, required: true },
  timeSlot: { type: String, required: true },
  status: {
    type: String,
    enum: ['Scheduled', 'Completed', 'Cancelled'],
    default: 'Scheduled'
  },
  symptoms: { type: String },
  notes: { type: String }
}, { timestamps: true });

module.exports = mongoose.model('Appointment', appointmentSchema);
```

6. Key REST API Endpoints Specification

HTTP Method	Endpoint	Access Role	Description
POST	<code>`/api/v1/auth/register`</code>	Public / Admin	Register new patient or staff member.
POST	<code>`/api/v1/auth/login`</code>	Public	Authenticate user and return JWT token.
GET	<code>`/api/v1/appointments`</code>	Admin / Staff	Get list of all appointments.
POST	<code>`/api/v1/appointments/book`</code>	Patient / Receptionist	Book a new appointment slot.
POST	<code>`/api/v1/prescriptions`</code>	Doctor	Create prescription and diagnosis entry.
GET	<code>`/api/v1/prescriptions/patient/:id`</code>	Doctor / Patient / Admin	Fetch medical history for a specific patient.

7. Security & Best Practices

- **Password Hashing:** All user passwords are encrypted using `bcryptjs` with salt rounds before database storage.
- **JWT Access Controls:** Stateless authentication using JSON Web Tokens passed in Authorization headers or HTTP-only cookies.
- **Input Validation:** Sanitization and payload validation using `express-validator` to prevent SQL/NoSQL Injection and XSS attacks.
- **CORS & Security Headers:** Configured `cors` middleware restricting origins and `helmet` for secure HTTP response headers.

8. Deployment & Operational Setup

The MERN Hospital Management System can be deployed using modern cloud platforms:

- **Database:** MongoDB Atlas (Cloud Managed Cluster).
- **Backend API:** Node.js server hosted on AWS EC2, Render, or Railway.
- **Frontend Client:** Static React build deployed on Vercel or Netlify.
- **Environment Setup:** Production deployment managed via Docker containers with Nginx reverse proxying.